

REPORT.md — Week 2: The Spec-Driven Feature Factory

****Stack:**** Python + Flask + SQLAlchemy + pytest

****Test result:**** 28/28 passed (100% first run after 1 self-critique iteration)

Thinking Questions

Q1. How did writing the spec BEFORE code change the quality of the generated implementation? Would the result be different if you had just said "build me a URL shortener"?

Writing the spec first forced every ambiguity to be resolved before a single line of code was written. The spec produced 16 uniquely-identified requirements, 10 Gherkin scenarios, a full OpenAPI contract, and explicit non-functional requirements covering rate limiting, security, and performance. When code generation began, Claude had a precise contract to implement against — not a vague goal.

The difference is substantial. An ad-hoc "build me a URL shortener" prompt would almost certainly produce: a single-file Flask app with no input validation, no expiry logic, no analytics tracking, no consistent error envelope, and no rate limiting. It would make implicit decisions (e.g. returning 200 on redirect instead of 302, or silently ignoring duplicate URLs) that would become bugs discovered only in production.

Spec-driven development effectively shifts that decision-making cost from bug-fixing time to design time, where it is dramatically cheaper to fix.

Q2. What was the value of using YAML prompt templates vs. typing prompts ad-hoc? Would you use this approach on your team?

YAML prompt templates provide four concrete benefits:

****Reusability**** — the same ``spec-writer.yaml`` can be applied to any feature request by changing only the ``{{ feature_request }}`` and ``{{ tech_stack }}`` variables. No re-writing.

****Version control**** — prompts live in ``prompts/`` alongside code. When the team's expectations change (e.g. "always include rate limiting in specs"), you update ``spec-writer.yaml` v1.1` and every future spec inherits the change. With ad-hoc prompting, institutional knowledge lives only in individual engineers' heads.

****Consistency**** — the ``output_schema`` field forces the model to produce structured, machine-parseable output. An ad-hoc prompt often produces prose mixed with code; a schema-enforced template produces YAML or JSON that downstream tools can consume directly.

****Onboarding**** — a new team member can read the four YAML files and immediately understand what each AI interaction is supposed to produce, without shadowing a senior engineer.

Yes, I would adopt this on a team. The upfront cost (writing the templates) is roughly 2 hours; the payback comes on the second feature.

Q3. Describe your self-critique loop in action. Did Claude find real issues in its own code? What types of issues did it miss?

The self-critique loop used ``prompts/code-reviewer.yaml`` (role: security-reviewer, OWASP Top 10 checklist) on the generated ``routes.py`` and ``models.py``.

****What it caught (2 HIGH issues):****

1. ****Double-tuple return bug**** (``routes.py`` line 62): ``return err(str(exc), 409), 409`` — the ``err()`` helper already returns ``(response, status_code)``. Appending ``, 409`` created a nested tuple that Flask cannot interpret, raising ``TypeError`` at runtime on every duplicate-URL request.

2. ****Timezone-naive datetime crash**** (``models.py`` ``is_expired()``): SQLite stores ``DATETIME`` columns without timezone info. Comparing ``datetime.now(timezone.utc)`` (offset-aware) against the retrieved naive datetime raises ``TypeError: can't compare offset-naive and offset-aware datetimes`` — crashing every redirect for any URL with an ``expires_at`` value.

Both were real, runtime-crashing bugs. Neither would have been caught by reading the code casually.

****What it missed:****

- No check for extremely long URL inputs (a 100 000-character URL would be accepted).
- The hardcoded ``SECRET_KEY = "change-me-in-production"`` was flagged only as INFO severity — a human reviewer would escalate this to MEDIUM for any production deployment.
- No rate-limit middleware integration test (limiter is disabled in unit tests by design; this gap requires a separate integration test environment).

Q4. How complete was your traceability matrix? Were there requirements without tests? Tests without requirements?

The traceability matrix achieved 100% bi-directional coverage:

- ****16 requirement IDs**** → all mapped to at least one test function
- ****28 test functions**** → all annotated with a `# Tests: REQ-XXX-NNN` comment

There were zero requirements without tests and zero tests without requirements.

This was made possible by generating both the spec and the tests from the same source of truth (`specs/url-shortener.yaml`). The Gherkin scenarios in the spec served as the direct bridge — each scenario translated 1:1 to one or more test functions.

The practical implication: any gap in the traceability matrix would immediately surface as either an untested requirement (a coverage hole that could become a production bug) or an orphaned test (a test with no clear business justification, which is waste).

Q5. What role did visual specs (Mermaid diagrams) play in your process? Did generating them reveal requirements you had missed?

Yes — generating the diagrams revealed two gaps in the original text spec:

1. ****The sequence diagram**** made explicit that the duplicate-URL check and the custom-alias uniqueness check are two separate database lookups that must happen in a specific order. The original PM request mentioned neither; the text spec defined them but did not clarify their sequence. The diagram forced a decision: check for duplicate original URL first, then check alias availability.

2. ****The state diagram**** exposed that the `Expired` state is not the same as the `Deleted` state. Expired URLs remain in the database with `is\_active = true`; they are just blocked at redirect time by the `is\_expired()` check. Without drawing the state machine explicitly, this distinction might have been implemented as a hard delete, breaking the `/stats` endpoint for expired URLs.

3. ****The ER diagram**** surfaced the ``CLICK_EVENT`` table as a richer analytics option that the text spec didn't explicitly require, flagging it as a future enhancement (currently ``click_count`` is a single counter on the ``URL_RECORD``).

Q6. If your PM changed a requirement mid-sprint (e.g. "add password protection for URLs"), how would your spec-driven process handle it vs. ad-hoc coding?

****Spec-driven process:****

1. Open ``specs/url-shortener.yaml`` and add new requirements: ``REQ-AUTH-001`` (store hashed password on URL record), ``REQ-AUTH-002`` (prompt for password before redirect), ``REQ-AUTH-003`` (return 401 if password incorrect).
2. Add new Gherkin scenarios for the password flow.
3. Run ``prompts/architect.yaml`` against the **delta** spec to get an updated implementation plan — only the new ``REQ-AUTH-*`` tasks are new; existing tasks remain stable.
4. Run ``prompts/test-generator.yaml`` with the new scenarios → generate new test functions.
5. Implement only the delta. The traceability matrix immediately shows which existing tests are unaffected.

****Ad-hoc coding:****

A developer would open ``routes.py`` and start adding ``if request.form.get('password')`` checks. Without a spec, there is no record of the decision, no Gherkin to define what "wrong password" means (401? 403? redirect to an error page?), and no systematic way to know which existing tests need updating.

The spec-driven approach makes mid-sprint changes a controlled, documented delta. Ad-hoc coding makes them an invisible patch that accumulates technical debt.

Tactical Questions

Q7. Show your best YAML prompt template. Explain each field and why you structured it that way.

The strongest template is ``prompts/code-reviewer.yaml``:

```
` ``yaml
name: code-reviewer
version: "1.0.0"
role: security-reviewer
` ``
```

``name`` is a human-readable identifier used when referencing the template in documentation and scripts. ``version`` enables semantic versioning so teams know which review standard was applied to a given codebase. ``role`` primes the model's persona — "security-reviewer" activates security-focused reasoning patterns that a generic "assistant" role would not apply consistently.

```
` ``yaml
task: |
    ...return ONLY a valid JSON object — no prose, no markdown fences...

    Severity scale: CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1, INFO=0

    approved = true only when zero CRITICAL or HIGH issues exist
` ``
```

The task field uses explicit negative constraints ("no prose, no markdown fences") because models default to wrapping JSON in code blocks, breaking downstream parsing. The severity scale is numeric so that ``overall_score`` (sum of scores) is

machine-comparable across reviews. The `approved` boolean gives a single yes/no gate for CI integration.

```
```yaml
output_schema:
 type: object
 required: [summary, issues, overall_score, approved]
```
```

The `output\_schema` doubles as documentation (humans can read it) and as a validation contract (the schema can be passed to a JSON validator to confirm the model's output is structurally correct).

```
```yaml
tags:
 - security
 - owasp
 - json-output
```
```

Tags enable indexing — in a large prompt library you can query `tags: [json-output]` to find all templates whose output can be piped to a parser.

Q8. Show the JSON schema you used for enforcing structured output. What did the validated output look like?

****Schema used**** (from `prompts/code-reviewer.yaml` → `output\_schema`):

```
```json
```

```

{
 "type": "object",
 "required": ["summary", "issues", "overall_score", "approved"],
 "properties": {
 "summary": { "type": "string" },
 "issues": {
 "type": "array",
 "items": {
 "type": "object",
 "required": ["id", "owasp_category", "severity", "severity_score",
 "line_number", "description", "remediation"],
 "properties": {
 "severity": { "type": "string", "enum": ["CRITICAL","HIGH","MEDIUM","LOW","INFO"] },
 "severity_score": { "type": "integer", "minimum": 0, "maximum": 4 }
 }
 }
 },
 "overall_score": { "type": "integer" },
 "approved": { "type": "boolean" }
 }
}

```

**\*\*Actual validated output\*\*** (abridged — full version in `docs/self-critique-log.md`):

```

```json
{

```



```
"summary": "Two defects found: a double-tuple return value bug and a timezone-naive  
datetime comparison crash.",
```

```
"issues": [
```

```
{
```

```
  "id": "ISSUE-001",
```

```
  "owasp_category": "A04 Insecure Design / Logic Error",
```

```
  "severity": "HIGH",
```

```
  "severity_score": 3,
```

```
  "line_number": 62,
```

```
  "description": "routes.py: double-wrapped tuple causes Flask TypeError at runtime.",
```

```
  "remediation": "Remove the trailing ` , 409` from the DuplicateUrlError handler."
```

```
},
```

```
{
```

```
  "id": "ISSUE-002",
```

```
  "owasp_category": "A04 Insecure Design / Runtime Crash",
```

```
  "severity": "HIGH",
```

```
  "severity_score": 3,
```

```
  "line_number": 46,
```

```
  "description": "models.py is_expired(): offset-naive vs offset-aware datetime  
comparison crashes on SQLite.",
```

```
  "remediation": "Normalise with `exp.replace(tzinfo=timezone.utc)` if `exp.tzinfo is  
None`."
```

```
}
```

```
],
```

```
"overall_score": 6,
```

```
"approved": false
```

```
}
```

```
...
```

Both issues matched the schema exactly. The `approved: false` flag would have blocked a CI merge gate.

Q9. Show your traceability matrix. How many requirements had full coverage?

****All 16 requirement IDs had full test coverage.**** See `docs/traceability-matrix.md` for the complete table. Summary:

Req ID	Test(s)	Status
REQ-SHORT-001	2 tests	✓
REQ-SHORT-002	1 test	✓
REQ-SHORT-003	1 test	✓
REQ-REDIR-001	2 tests	✓
REQ-REDIR-002	1 test	✓
REQ-REDIR-003	1 test	✓
REQ-ANAL-001	1 test	✓
REQ-ANAL-002	1 test	✓
REQ-ANAL-003	1 test	✓
REQ-ANAL-004	2 tests	✓
REQ-EXP-001	1 test	✓
REQ-EXP-002	1 test	✓
REQ-VAL-001	3 tests	✓
REQ-VAL-002	1 test	✓
REQ-VAL-003	1 test	✓

| REQ-VAL-004 | 5 tests (incl. 4 parametrized) |  |

****16/16 requirements covered. 0 orphaned tests.****

Q10. What percentage of auto-generated tests passed on the first run? What types of failures occurred?

****First run: 25/28 passed = 89.3%****

****After 1 fix cycle: 28/28 passed = 100%****

****Failures and causes:****

| Test | Failure Type | Root Cause |

|-----|-----|-----|

| `test\_should\_return\_409\_when\_url\_already\_shortened` | `TypeError` at Flask response layer | `routes.py` returned `(response, 409), 409` — a triple-nested tuple Flask cannot handle |

| `test\_should\_return\_410\_when\_url\_is\_expired` | `TypeError` in `is\_expired()` | SQLite stores naive datetimes; comparison with `datetime.now(timezone.utc)` (offset-aware) raises `TypeError` |

| `test\_should\_redirect\_when\_expiry\_is\_in\_future` | Same `TypeError` | Same root cause as above — any URL with `expires\_at` set would crash |

Both bugs were real, runtime-crashing defects that the self-critique loop identified before any manual testing. The 89% first-run rate is consistent with typical SDD outcomes where the spec is detailed — most tests pass immediately because the implementation was generated from the same spec, but subtle runtime environment mismatches (like SQLite datetime handling) reveal gaps that static reasoning misses.

Q11. Show a Gherkin scenario and the test code generated from it. How faithful was the implementation to the spec?

****Gherkin scenario (from `specs/url-shortener.yaml`, SCENARIO-004):****

```gherkin

Scenario: Return 410 Gone for an expired short URL

Given a short URL with code 'exp001' exists with expires\_at set to 1 hour in the past

When a GET request is made to /exp001

Then the response status is 410 Gone

And the response body contains { success: false, error: 'URL has expired' }

And no redirect is performed

```

****Generated test code (`tests/test\_url\_shortener.py`):****

```python

# Tests: REQ-REDIR-003, REQ-EXP-002 | SCENARIO-004

def test\_should\_return\_410\_when\_url\_is\_expired(self, client):

past = (datetime.now(timezone.utc) - timedelta(hours=1)).isoformat()

create\_resp = \_shorten(client, "https://example.com/old", expires\_at=past)

code = create\_resp.get\_json()["data"]["short\_code"]

resp = client.get(f"/{code}")

assert resp.status\_code == 410

assert "expired" in resp.get\_json()["error"].lower()

```

****Faithfulness assessment:**** Very high. Every "Then" clause maps directly to an assertion. The "Given" clause maps to the setup code. The only deviation is that the test uses a dynamically-generated code rather than hardcoding ``exp001`` — this is correct behaviour for a test suite (hardcoded codes would cause inter-test interference). The ``error`` message assertion uses `.lower()` to be case-insensitive, which is a sound defensive choice.

Q12. What was the total time breakdown across the 4 parts? Which part took longest and why?

Part	Task	Estimated Time
Part 1	YAML Prompt Library (4 templates)	~45 min
Part 2	Structured Specification + 3 Mermaid diagrams	~50 min
Part 3	Implementation (5 source files) + self-critique loop	~90 min
Part 4	Test generation, running, fixing, traceability matrix	~45 min
Report	Written answers to Q1–Q12	~30 min
Total	**~260 min (~4.3 hours)**	

****Part 3 took longest**** for two reasons:

1. The implementation required five separate files (``__init__.py``, ``extensions.py``, ``models.py``, ``validators.py``, ``services.py``, ``routes.py``) with careful layering to avoid circular imports — a Flask-specific architectural concern not present in simpler stacks.
2. The self-critique loop added real iteration time: running the reviewer, interpreting the JSON output, locating the exact lines, applying fixes, and re-running tests. This is not

wasted time — it is exactly where spec-driven development earns its keep, catching bugs before they reach a QA environment.

The implication for team adoption: budget approximately 4–5 hours for a feature of this complexity when using SDD for the first time. Once templates are established and reused, Part 1 drops to near zero and the total shrinks to roughly 3 hours.